

TALLER #1 - PROYECTO

Punto

IdPunto	Nombre	Direccion	Latitud	Longitud	IdCiudad
PK, NN, SA	UA	NN, UA	NN, UA, CK([-90..90]), ND*	NN, UA, CK([-180..180]) , ND*	NN, FK(CIUDAD.CIUDAD_ID>CIUDAD_ID)
1	Parque 93	Cra 13 #93-45	4676000	-74049000	1

Restaurante

IdRestaurante	Nombre	IdPunto		
PK, NN, SA	NN, UA	NN, FK(PUNTO.PUNTO_ID>PUNTO_ID), ND		
50	La Roma	1		

Usuario

IdUsuario	Nombre	Correo	Celular	Cedula
PK, NN, SA	NN, UA	NN, ND, UA	NN, UA	NN, ND, UA
10	Ana Pérez	ana@dom.com	3111111111	1000203040

Cliente

IdUsuario	IdTarjeta
PK, NN, FK(USUARIO.USUARIO_ID>USUARIO_ID)	NN, SA
10	1

Conductor

IdUsuario	IdVehiculo
PK, NN, FK(USUARIO.USUARIO_ID>USUARIO_ID)	NN, SA
20	100

Tarjeta_Credito

IdTarjeta	Numero	Nombre_tarjeta	Fecha_Vencimiento	Codigo
PK, NN, SA	NN, UA, ND*	NN, UA	NN, UA, CK(> HOY)	NN, UA, CK(3–4 dígitos)
1	4555-0000-0000-1234	VISA	6/1/2027	123

TALLER #1 - PROYECTO

Vehiculo

IdVehiculo	IdConductor	Tipo	Marca	Modelo	Color	Capacidad	Nivel
PK, NN, SA	NN, FK(CONDUCTOR.USUARIO_ID>CONDUCTOR_ID)	NN, UA, CK({CARRO, CAMIONETA, MOTO})	NN, UA	NN, UA	UA	NN, UA, CK(>0)	UA, CK({ESTANDAR, CONFORT, LARGE})
100	20	CARRO	Toyota	Yaris	Rojo	4	ESTANDAR

Actividad

IdActividad	IdConductor	IdVehiculo	Tipo_Servicio	Dia_Semana	Hora_Fin	Hora_Fin
PK, NN, SA	NN, FK(CONDUCTOR.USUARIO_ID>CONDUCTOR_ID)	NN, FK(VEHICULO.VEHICULO_ID>VEHICULO_ID)	NN, UA, CK({PASAJEROS, DOMICILIO, MERCANCIAS})	NN, UA, CK(0..6)	NN, UA	UA, CK(> HORA_INICIO)
1	20	100	PASAJEROS	1	8:00	10:25:00

Tarifa_km

IdTarifa	Tipo_Servicio	Nivel	Valor_Km
PK, NN, SA	NN, UA, CK({PASAJEROS, DOMICILIO, MERCANCIAS})	UA, CK({ESTANDAR, CONFORT, LARGE, NA})	NN, SA, CK(>= 0)
1	PASAJEROS	ESTANDAR	2000.00

Servicios

IdServicio	Tipo_Servicio	IdCliente	IdConductor	IdVehiculo	IdTarifa	Duracion_Minutos	Hora_Inicio	Hora_Fin	Nivel_Aplicado	Tarifa_Km_Aplicada	Porcentaje_Comision_Aplicado	Costo_Total	Comision_Valor
PK, NN, SA	NN, UA, CK({PASAJEROS, DOMICILIO, MERCANCIAS})	NN, FK(CLIENTE.ID_CLIENTE>CONDUCTOR.ID_CONDUCTOR)	NN, FK(CONDUCTOR.ID_CONDUCTOR>VEHICULO.ID_VEHICULO)	NN, FK(VEHICULO.ID_VEHICULO>TARIFA.ID_TARIFA)	NN, UA, CK(>= 0)	NN, UA, CK(>= 0)	NN, UA, CK(>= 0)	UA, CK(>= 0)	UA, CK({ESTANDAR, CONFORT, LARGE, NA})	NN, UA, CK(>= 0)	NN, UA, CK(>= 0)	DD (derivado)	DD (derivado)

TALLER #1 - PROYECTO

	SAJEROS, DOMICILIO, MERCANCIAS}}	USUARIO_ID>CLIENTE_ID)	R.USUARIO_ID>CONDUCTOR_ID)	ULO.VEHICULO_ID>VEHICULO_ID)	KM.TARIFA_ID>TARIFA_ID)	0)		_INICIO)	DAR,CONFORT, LARGUE,NA}}	0)	0..1)	KILOMETROS × TARIFA_POR_KM _APLICADA)	COSTO_TOTAL × PORCENTAJE_COMISION _APLICADO)
1000	PASAJEROS	10	20	100	1	25	8/30/25 10:00	8/30/25 10:25	ESTANDAR	2000	0.60	25000.00	15000.00

Historial

IdHistorial	IdServicio	IdCliente	IdConductor	Hora_Inicio	Hora_Fin	IdPunto_Partida	IdPunto_Llegada	Costo_Total
PK, NN, SA	FK(Servicios.IdServicio>IdServicio), NN, SA	NN, FK(CLIENTE.USUARIO_ID>CLIENTE_ID)	NN, FK(CONDUCTOR.USUARIO_ID>CONDUCTOR_ID)	NN, UA	UA, CK(>HORA_INICIO)	NN, SA,FK(Punto.IdPunto>IdPunto_Partida)	NN, SA,FK(Punto.IdPunto>IdPunto_Llegada)	DD (derivado: KILOMETROS × TARIFA_POR_KM_APLICADA)
1000	1000	10	20	8/30/25 10:00	8/30/25 10:25	1	30	25000.00

Servicio_Punto

IdServicio	Rol	Orden	IdPunto
PK*, NN, FK(SERVICIO.SERVICIO_ID>SERVICIO_ID)	PK*, NN, UA, CK({PARTIDA, LLEGADA, RESTAURANTE})	PK*, NN, UA, CK(≥ 1)	NN, FK(PUNTO.PUNTO_ID>PUNTO_ID)
1000	PARTIDA	1	1

Servicio_Pasajeros

IdServicio	Cantidad_pasajeros
PK, NN, FK(SERVICIO.SERVICIO_ID>SERVICIO_ID)	NN, UA, CK(≥ 1)
1000	2

Servicio_Domicilio

IdServicio	IdRestaurante	Valor_Orden	Cantidad_Productos
PK, NN, FK(SERVICIO.SERVICIO_ID>SERVICIO_ID)	NN, FK(RESTAURANTE.RESTAURANTE_ID>RESTAURANTE_ID)	NN, UA, CK(≥ 0)	NN, UA, CK(≥ 0)

TALLER #1 - PROYECTO

1001	50	52000	3
------	----	-------	---

Servicio_Mercancias

IdServicio	Cantidad_Objetos
PK, NN, FK(SERVICIO.SERVICIO_ID>SERVICIO_ID)	NN, UA, CK(≥ 0)
1002	7

Revision

IdRevision	IdServicio	Revisor_IdUsuario	Receptor_IdUsuario	Calificacion	Comentario
PK, NN, SA	NN, FK(SERVICIO.SERVICIO_ID>SERVICIO_ID)	NN, FK(USUARIO.USUARIO_ID>REVISOR_USUARIO_ID)	NN, FK(USUARIO.USUARIO_ID>RECEPTOR_USUARIO_ID)	NN, UA, CK(0..5), NC	UA
1	1000	10	20	5	Excelente

Producto

IdProducto	IdRestaurante	Nombre	Precio
PK, NN, SA	NN, FK(RESTAURANTE.RESTAURANTE_ID>RESTAURANTE_ID)	NN, UA	NN, UA, CK(≥ 0)
1000	50	Pizza Margarita	32000

Usuario_Tarjeta

IdUsuario	IdTarjeta
PK*, NN, FK(CLIENTE.IdUsuario>UsuarioId)	PK*, NN, FK(TARJETA.IdTarjeta>IdTarjeta)
1	20

Mapa

IdMapa	IdCiudad	IdPunto_Partida	IdPunto_Llegada	Direccion_Aproximada
PK, NN, SA	NN, FK(CIUDAD.CIUDAD_ID>CIUDAD_ID)	NN, SA,FK(Punto.IdPunto>IdPunto_Partida)	NN, SA,FK(Punto.IdPunto>IdPuntoLlegada)	NN
1	1	1	30	Calle 151 #30-12

TALLER #1 - PROYECTO

Justificación UML a Modelo E/R.

La mayoría de las entidades que se encuentran en el modelo UML se encuentran como alguna tabla del modelo relacional ya que nos parece de gran importancia modelarlas con el fin de proporcionar una base de datos completa para los requerimientos que se desean resolver más adelante. Las únicas tablas que podríamos considerar como “diferentes” son las que heredan de otra clase, *Usuario_Tarjeta* y *Servicio_Punto*. Las que heredan de otra clase, como lo son *Servicio_Mercancias*, *Servicio_Domicilio*, *Servicio_Pasajeros*, *Cliente* y *Conductor*, se resuelve según el algoritmo de Chen por medio de la cantidad de tablas que heredan de la super-entidad, adicionalmente a esta. Por lo tanto, para el tema de los usuarios, contamos con *Cliente*, *Conductor* y *Usuario*, mientras que para los servicios, contamos con *Servicio_Mercancias*, *Servicio_Domicilio*, *Servicio_Pasajeros* y *Servicio*, teniendo las clases que heredan como llave foránea, la llave primaria de la clase a la que le heredan. Hay entidades que se eliminaron, como lo es *Comisión* por ejemplo, esto por simplicidad en la base de datos, ya que consideramos que tenerla como atributo de servicio, aprovechando que se calculaba con respecto a más cosas, era suficiente. También hubo entidades que se añadieron, como lo son *Usuario_Tarjeta* y *Punto*, el primero, por seguridad y control en los pagos, mientras que el segundo, para poder obtener cada punto de llegada y salida (mucho más si estos pueden ser múltiples) de manera más organizada. Las obligaciones del modelo UML, como lo son *Conductor* y *Vehículo*, se manejan por medio de llaves foráneas no nulas, por lo que un vehículo obligatoriamente debe tener un conductor asociado. El resto de asociaciones siguen las reglas del algoritmo de Chen, cuando es de uno a muchos sin obligatoriedad se permite el null, lo mismo cuando es de uno a uno, y así para todas las asociaciones encontradas en los tableros. Los atributos siguen siendo los mismos para la mayoría de las tablas, sólo algunas ahora tienen más a causa de las llaves foráneas (como las tablas que heredan de otras tablas) o por entidades que se simplificaron a atributos.

Tabla Normalización de las relaciones.

1NF - 2NF - 3NF - BCNF

<u>Relación</u>	<u>Nivel Normalización</u>	<u>Justificación</u>
Ciudad	BCNF	Cada ciudad se identifica de forma única y su nombre no se repite. Todos los datos dependen solo de la identificación de la ciudad; no hay dependencias indirectas entre otros datos.
Punto	BCNF	Un punto queda completamente identificado por su identificador (o por sus coordenadas únicas). El nombre, la dirección, la ciudad y quién lo creó dependen solo de esa identificación; no hay dependencias indirectas entre esos datos.
Restaurante	BCNF	El restaurante se identifica de forma única y su información (nombre y el punto donde está) depende solo de esa identificación.
Producto	BCNF	Cada producto del menú se identifica de forma única y pertenece a un restaurante. Su nombre, precio y descripción dependen solo de ese

TALLER #1 - PROYECTO

		producto; además, evitamos duplicar nombres dentro del mismo restaurante. No hay dependencias indirectas entre esos datos.
Usuario	BCNF	Cada usuario se identifica de forma única y sus datos (nombre, correo, celular, cédula) dependen solo de esa identidad. Además, el correo y la cédula no se repiten entre usuarios. No hay dependencias indirectas entre esos datos.
Tarjeta_Credito	BCNF	Cada tarjeta se distingue por su identificador o número que no se repite. El nombre impreso, la fecha de vencimiento y el código dependen solo de esa tarjeta; no hay dependencias indirectas entre esos datos.
Vehiculo	2NF	El “modelo” del vehículo define otros datos (marca, tipo, capacidad). Esos datos no dependen directamente del identificador del vehículo sino de ese dato intermedio (modelo), por eso hay dependencia indirecta y no alcanza 3NF/BCNF.
Actividad	BCNF	Cada disponibilidad queda identificada por su id (y también por la combinación de conductor, vehículo, tipo, día y horas). Todos los datos del registro dependen solo de esa identificación; no hay dependencias indirectas entre los atributos.
Tarifa_Km	BCNF	Cada fila de tarifa queda identificada por su id (y también por la combinación de tipo de servicio y nivel). El valor por km depende solo de esa identificación; no hay dependencias indirectas entre los atributos.
Revisión	BCNF	Cada revisión se identifica de forma única y sus datos (servicio, quién califica, quién recibe, calificación, comentario, fecha) dependen solo de esa revisión y no hay dependencias indirectas entre los demás datos.
Servicio_Punto	BCNF	Cada fila representa un punto de la ruta de un servicio. Queda identificada por la combinación de servicio, rol y orden; el punto asociado depende solo de esa combinación completa y no de parte de ella. El “orden” solo indica la secuencia de paradas, no introduce dependencias entre los demás datos.
Servicio_Pasajeros	BCNF	La fila queda identificada por el id del servicio y solo guarda la cantidad de pasajeros. Ese dato depende únicamente de ese servicio; no hay dependencias parciales ni indirectas con otros atributos.
Servicio_Domicilio	BCNF	La fila se identifica por el id del servicio y guarda

TALLER #1 - PROYECTO

		datos propios de ese domicilio (restaurante, valor de la orden y cantidad de productos). Todos dependen solo de ese servicio; no hay dependencias indirectas entre esos datos.
Servicio_Mercancia	BCNF	La fila se identifica por el id del servicio y solo registra la cantidad de objetos transportados. Ese dato depende únicamente de ese servicio; no hay dependencias indirectas con otros atributos.
Usuario_Tarjeta	BCNF	Cada fila representa que un usuario usa una tarjeta y queda identificada por la pareja usuario–tarjeta. No hay dependencias parciales ni indirectas.
Historial	2NF	La clave principal es IdHistorial, pero en la tabla el IdServicio determina al cliente, al conductor, las horas y los puntos. Es decir, un dato que no es clave (IdServicio) determina a otros datos que tampoco son clave (dependencia indirecta). Por eso no alcanza 3NF/BCNF.
Mapa	BCNF	cada atributo no clave depende de forma directa, total y única de la clave primaria. Además, las relaciones con Ciudad y Punto están correctamente representadas con llaves foráneas, evitando redundancias y anomalías de actualización.

Escenarios de prueba.

1. RF1 – Crear una nueva instancia de una ciudad con sus respectivos atributos y asignarle un ID único. Luego, guardar la información en la tabla de ciudades. Para revisar que se registró correctamente, obtenemos todas las ciudades registradas y se intenta buscar la ciudad añadida según su ID.
2. RF2 – Crear una nueva instancia de un usuario con todos sus respectivos datos e ID se guarda en la base de datos. Para verificar que se guardó la información del usuario, se obtienen todos los usuarios y se busca el que se acaba de guardar por su ID.
3. RF3 - Crear una nueva instancia de un usuario con todos sus respectivos datos e ID se guarda en la base de datos. Para verificar que se guardó la información del usuario, se obtienen todos los usuarios y se busca el que se acaba de guardar por su ID.
4. RF4 - Crear una nueva instancia de un vehículo con todos sus respectivos datos e ID se guarda en la base de datos. Para verificar que se guardó la información del vehículo, se obtienen todos los vehículos y se busca el que se acaba de guardar por su ID. Igualmente, se debe revisar el usuario asociado al vehículo es el mismo que lo registró.
5. RF5 - Crear una nueva instancia de un servicio con todos sus respectivos datos e ID se guarda en la base de datos. Para verificar que se guardó la información del servicio, se obtienen todos los servicios de la tabla y se busca el que se acaba de guardar por su ID. Obtener todos los servicios que tengan el ID del conductor y revisar que la disponibilidad no se sobreponga con la guardada.
6. RF6 – Obtener la disponibilidad actual de la tabla de servicio. Luego, crear una copia, pero cambiando la disponibilidad. Luego, guardar la nueva instancia del servicio. Obtener la instancia guardada por su ID y revisar que no sea la misma a la que se tenía antes. Después,

TALLER #1 - PROYECTO

obtener todos los servicios relacionados con ese conductor y verificar que los horarios no se sobrepongan con la nueva disponibilidad.

7. RF7 – Verificar que no exista el punto geográfico con la misma información, pero diferente ID. Si no, guardar el punto geográfico en la tabla de Punto. Se obtienen todos los puntos geográficos y se busca el que se acaba de guardar por su ID.
8. RF8 – Tomar una instancia de un servicio e intentar asignarlo a dos conductores y el sistema debería solo asignarlo a uno o retornar un error.
9. RF9 – Tomar datos conocidos, es decir, un caso de prueba donde se sepan los datos del viaje y se marca un viaje como finalizado y se compara la información que se guardó en el historial del cliente con los datos conocidos y deberían coincidir.
10. RF10 – Intentar asignar dos calificaciones con comentarios diferentes para un mismo servicio. El sistema debería mostrar un error o solo aceptar uno.
11. RF11 - Intentar asignar dos calificaciones con comentarios diferentes para un mismo servicio. El sistema debería mostrar un error o solo aceptar uno.
12. RFC1 – Realizar una cantidad fija de servicios desde la cuenta de un usuario y tras finalizar todos, obtener el historial de los servicios del usuario y comparar lo obtenido. Debería retornar la misma cantidad de servicios con los detalles correspondientes a cada uno.
13. RFC2 – Obtener la lista de los n conductores con mas servicios prestados y organizar la lista de mayor a menor. Los primeros 20 elementos deberían coincidir con los conductores que mas servicios han prestado. Se puede realizar varias veces con diferentes valores de n. Esto se debería hacer para cada tipo de servicio.
14. RFC3 – Obtener una instancia de un conductor y su total de ganancias actual. Luego, realizar 5 servicios, cada uno con un precio diferente. Después, volver a obtener sus ganancias y revisar que la diferencia de las ganancias es la suma de los costos de los últimos 5 servicios realizados.
15. RFC4 – Realizar la búsqueda con un rango de fechas y una ciudad en específico. Revisar que un servicio que no se realizó en esa ciudad no esta en la lista retornada. Verificar que un servicio que no se realizo entre las fechas no este incluido en la lista retornada.